

Face Recognition using SVD

Davide Settecasì

May 10, 2026

Abstract

Write here what the report is about, what you did, and what you found. This should be a concise summary of the entire report, highlighting the main objectives, methods, results, and conclusions.

Contents

1	Introduction	1
2	Singular Value Decomposition and Principal Component Analysis	1
2.1	SVD and low-rank approximation	2
2.2	Variance and PCA	2
2.3	Eigenfaces	3
3	Experiments with the Yale Face Database	4
3.1	Face recognition	4
3.1.1	Identifying the optimal number of features	4
3.1.2	Evaluation of model robustness against unknown subjects	7
3.1.3	Leave- k -Out Cross-Validation for the model	8
3.2	Face reconstruction	10
3.3	Signal and noise in facial images	10

1 Introduction

Brief description of the problem, motivation, and objectives of the report. Explain why dimensionality reduction and image reconstruction are important topics in data analysis and computer vision.

2 Singular Value Decomposition and Principal Component Analysis

Let us briefly recall some basic properties of the Singular Value Decomposition (SVD). For simplicity, we will focus on the case of real matrices, but the SVD and the results we will discuss can be easily extended to the complex case. Given a matrix $A \in \mathbb{R}^{m \times n}$, we can decompose it as follows:

$$A = U \Sigma V^T$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix whose diagonal entries are $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ (where r is the rank of A) and the rest are zero. This factorization is called the Singular Value Decomposition of A and it can be proved that it always exists for any matrix A (cf. [1, Theorem 4.4.1, pp. 50–51]). The elements σ_i

are called the singular values of A , and the columns of U and V are called the left and right singular vectors, respectively. It can be seen by direct computation that the columns of U are eigenvectors of AA^T and the columns of V are eigenvectors of $A^T A$, while the singular values σ_i are the square roots of the non-zero eigenvalues of both AA^T and $A^T A$.

From a geometric point of view, the SVD can be interpreted as a rotation (given by V^T), followed by a scaling of the axes (given by Σ), and then another rotation (given by U). Let us denote the columns of U by $u_1, \dots, u_m$ and the columns of V by $v_1, \dots, v_n$. From this geometric interpretation, it can be seen that $Av_i = \sigma_i u_i$ for $i = 1, \dots, r$, and $Av_i = 0$ for $i > r$. This means that the right singular vectors v_i are mapped to the left singular vectors u_i by the matrix A , and the singular values σ_i represent the amount of stretching or compression along the corresponding singular vectors.

2.1 SVD and low-rank approximation

The SVD provides a powerful tool for low-rank approximation of matrices, that will be particularly useful in the context of dimensionality reduction and image reconstruction. From the geometric interpretation described above, it can be seen that the first left singular vectors correspond to the directions on which the matrix A stretches the most, while the last left singular vectors correspond to the directions on which the matrix A compresses the most. So, if we are interested in approximating A with a matrix of lower rank, it makes sense to keep only the first few singular values and the corresponding singular vectors, and set the rest to zero. This idea can be formalized through the following result, known as the Eckart-Young-Mirsky theorem (cf. [1, Theorem 4.4.5, pp. 53–54]):

Theorem 2.1. *Let $A \in \mathbb{R}^{m \times n}$ and $A = U\Sigma V^T$ be its SVD. Let A_k be defined as follows:*

$$A_k = U\Sigma_k V^T$$

where Σ_k is the diagonal matrix obtained from Σ by keeping only the first k singular values and setting the rest to zero. Then, the following hold true:

1. *The matrix A_k satisfies $\sigma_{k+1} \leq \|A - A_k\|_2 \leq \|A - B\|_2$ for any matrix B with rank less or equal to k .*
2. *The matrix A_k satisfies $\sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2} = \|A - A_k\|_F \leq \|A - B\|_F$ for any matrix B with rank less or equal to k .*

What this theorem states is that A_k is the best rank- k approximation of A with respect to both the Euclidean norm and the Frobenius norm.

An important quantity that can be used to measure the amount of information retained by the rank- k approximation A_k is the energy of the singular values, defined as follows:

$$E(k) = \frac{\sum_{j=1}^k \sigma_j^2}{\sum_{j=1}^r \sigma_j^2}.$$

If this quantity is close to 1, it means that the rank- k approximation A_k retains most of the information contained in A , while if it is close to 0, it means that A_k is a poor approximation of A .

2.2 Variance and PCA

Imagine now that we are given a dataset of n samples, each of which has m features. If m is large, some of the features might be correlated. For example, we have a dataset of real estate properties, and we have two features: the size of the property in square meters and the number

of rooms. These two features are likely to be correlated, since larger properties tend to have more rooms. In this case, it might be useful to reduce the dimensionality of the dataset by keeping only the most important features, that is, the features that retain most of the information contained in the original dataset. This is the main idea behind Principal Component Analysis (PCA), which is a technique for dimensionality reduction based on the SVD.

Now, we can represent the samples as n vectors in $\mathbb{R}^m$, $(x_i)_{i=1}^n$. We define the mean of the dataset as $a = \frac{1}{n} \sum_{i=1}^n x_i$, and we center the dataset by subtracting the mean from each sample, obtaining the centered dataset $(x_i - a)_{i=1}^n$. It is important to subtract the mean from the samples because we are interested in how the features vary with respect to the mean, and not in the absolute values of the features. Without loss of generality, we can assume that the dataset is already centered, so we will work with the centered dataset from now on.

We now want to measure how much the features vary with respect to the mean. To do this, we can compute the covariance matrix of the dataset, defined as follows:

$$C = \frac{1}{n} \sum_{i=1}^n x_i x_i^T = \frac{1}{n} X X^T$$

where X is the $m \times n$ matrix whose columns are the samples of the dataset. The diagonal entries of the covariance matrix represent the variance of each feature, that is, how much each feature varies with respect to the mean, while the off-diagonal entries represent the covariance between pairs of features, that is, how much two features vary together with respect to the mean. Since the covariance matrix is a symmetric positive semidefinite matrix, by spectral theorem it can be decomposed as

$$C = V_C \Lambda V_C^T$$

where V_C is an orthogonal matrix and Λ is a diagonal matrix whose diagonal entries are ordered in decreasing order $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \geq 0$. We are interested in the directions on which the features vary the most. We call these the principal components of the dataset. Formally, we want to find

$$\arg \max_{\|u\|_2=1} u^T C u = \arg \max_{\|u\|_2=1} u^T V_C \Lambda V_C^T u = \arg \max_{\|y\|_2=1} y^T \Lambda y$$

where we have made the change of variable $y = V_C^T u$ and we have used the fact that V_C is an orthogonal matrix. So we are interested in maximizing $\sum_{j=1}^m \lambda_j y_j^2$ subject to the constraint $\sum_{j=1}^m y_j^2 = 1$. By using the method of Lagrange multipliers, we can see that the maximum is attained when $y = e_1$. Since $y = V_C^T u$, we have that the direction on which the features vary the most is given by the first column of V_C , that is, the eigenvector of C corresponding to the largest eigenvalue λ_1 . By the same argument, we can see that the direction on which the features vary the second most is given by the eigenvector of C corresponding to λ_2 , and so on. So, if we want to reduce the dimensionality of the dataset by keeping only the most important features, we can keep only the first few eigenvectors of the covariance matrix.

We can express all this in terms of the SVD of the data matrix X . We recall that the left singular vectors of X are the eigenvectors of $X X^T$, so in order to compute the eigenvectors of the covariance matrix C , we can compute the SVD of $\frac{1}{\sqrt{n}} X$ and keep only the first few left singular vectors. The Eckart-Young-Mirsky theorem guarantees that this is the best way to reduce the dimensionality of the dataset while retaining as much information as possible.

2.3 Eigenfaces

We can apply the PCA technique discussed above to the problem of face recognition and compression of facial images, as illustrated in [2]. Assume that the facial images are represented as 2D arrays of size $m = p \times q$. We can represent each image as a vector in $\mathbb{R}^m$ by concatenating the rows of the image, so we have a dataset of n samples in $\mathbb{R}^m$. We observe that m is typically very

large (for example, if the images are of size 100×100 , we have $m = 10000$) while n is typically much smaller. We can ask ourselves if facial images are well approximated in a low-dimensional subspace of $\mathbb{R}^m$, or equivalently, in the language of the previous section, we want to find a small number of features that retain the most information about the facial images.

Given a dataset of facial images $(f_i)_{i=1}^n$, first we calculate the mean image $a = \frac{1}{n} \sum_{i=1}^n f_i$, and then we center the dataset by subtracting the mean image from each image, obtaining the centered dataset $(f_i - a)_{i=1}^n =: (x_i)_{i=1}^n$. We then introduce the data matrix $X = \frac{1}{\sqrt{n}}[x_1, x_2, \dots, x_n]$. From the previous section, we know that the principal components are the left singular vectors of X , so we can compute the SVD of X and keep only the first few left singular vectors. We call these left singular vectors the eigenfaces. Although the dimension of the original space is very large, usually the number ν of eigenfaces required in order to get a good low rank approximation is much smaller than m . So we can think that facial images are well approximated in a ν -dimensional subspace of $\mathbb{R}^m$ spanned by the eigenfaces.

We can now use the eigenfaces for our two main purposes: face recognition and face compression. For facial image compression, we project any centered facial image $f - a$ onto the subspace spanned by the eigenfaces. To compute the coordinates of the projection, we have to solve $U_\nu y = f - a$ in a least square sense, where U_ν is the matrix whose columns are the first ν left singular vectors of X . Since the columns of U_ν are orthonormal, we have that $y = U_\nu^T(f - a)$, and the projection of $f - a$ onto the subspace spanned by the eigenfaces is given by $U_\nu U_\nu^T(f - a)$.

For facial image recognition, imagine that we have already compressed the facial images in our dataset and that we are given a new face f . We compress the new face as illustrated above and then we look for the compressed face in our dataset that minimizes the distance with the compressed new face. If the distance is small enough, we can say that the new face is recognized as the same person of the compressed face in our dataset. We can use different kinds of distances to measure the distance between the compressed faces and we will experiment with different kinds of distances in the next sections.

3 Experiments with the Yale Face Database

In this section, we will apply the PCA technique described in the previous section to the Yale Face Database [3], which is a dataset of facial images that can be downloaded from <http://cvc.cs.yale.edu/cvc/projects/yalefaces/yalefaces.html>. It contains 165 grayscale images in GIF format of 15 individuals. There are 11 images per subject, one per different facial expression or configuration: center-light, w/glasses, happy, left-light, w/no glasses, normal, right-light, sad, sleepy, surprised, and wink. The code of the experiments can be found at INSERIRE INDIRIZZO GITHUB.

3.1 Face recognition

3.1.1 Identifying the optimal number of features

First we want to identify the optimal number of features for face recognition. To do so, we set up the following experiment. We select an expression, e.g. happy, and for each face in the dataset we exclude the selected expression from the training data. We then construct our data matrix X as described in the previous section and we compute the matrix U whose columns are the left singular vectors of X . Then we select the first p columns and we compute the matrix U_p whose columns are the first p left singular vectors of X . We project¹ the training data by computing $U_p^* X$ and we project the test data by computing $U_p^* X_{test}$. Then, for each face in the test data, we

¹We do not need to calculate the real projection $\Pi_{U_p} X = U_p U_p^* X$ because we are just interested in calculating Euclidean distances between the projected training data and the projected test data, so we can just left multiply by the matrix U_p^* , since U_p has orthonormal columns, i.e. it is an isometry, and the Euclidean distance is invariant under these transformations.

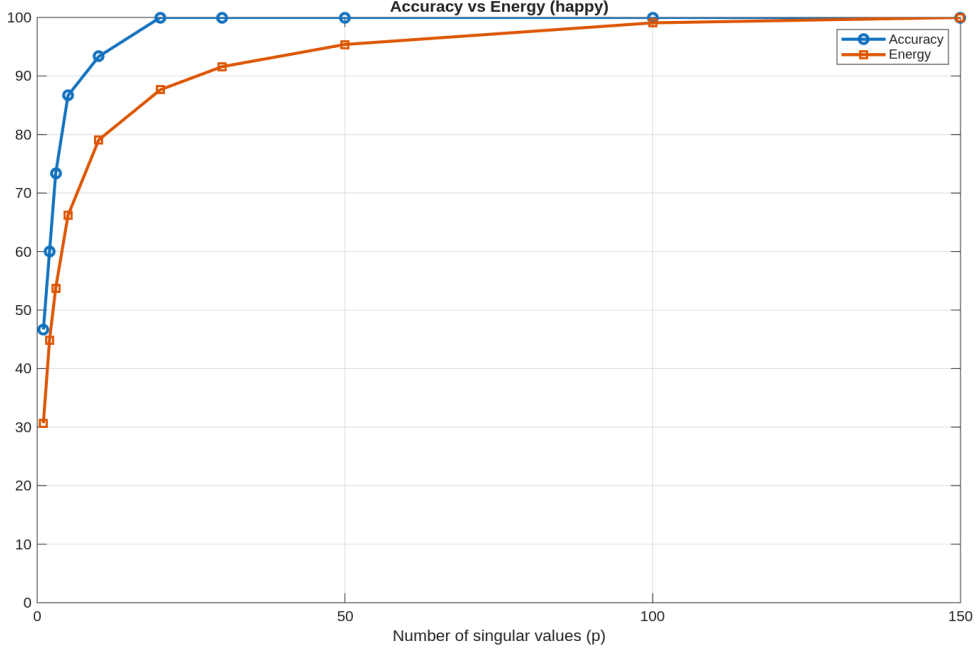


Figure 1: Accuracy and energy as a function of the number of features for happy expression.

compute the Euclidean distance from all the faces in the training data and we select the face that minimizes the distance. If the two faces have the same person, we say that the face recognition is successful. We repeat this experiment for different values of p (1, 2, 3, 5, 10, 20, 30, 50, 100, 150) and we plot the accuracy as a function of p . We also calculate the energy of the first p singular values and we plot also this quantity as a function of p . Results are shown in Figure 1.

We extended the experiment to other expressions, specifically 'sleepy' and 'left-light'. Figure 2 and figure 3 show the results for these two expressions. The results demonstrate that accuracy generally peaks rapidly with the initial features before reaching a plateau. For the sleepy expression, the algorithm achieves a perfect score (100%) with only $p = 10$ singular values, matching the energy saturation point. In contrast, for the happy expression, accuracy continues to improve more gradually, reaching a perfect score at $p = 20$. The left-light expression, however, exhibits a more complex and less efficient behavior. Accuracy reaches an initial peak of 33.33% at $p = 10$, but subsequently drops to 20% as the number of features increases to $p = 20$, remaining stagnant at this level for a significant range. It is only with a much higher number of features ($p = 150$) that the accuracy recovers to 33.33%.

Regarding the energy profile, we observe a steep increase within the first 10 singular values, reaching 80% of the total energy, followed by marginal gains. This suggests that choosing $p = 10$ or $p = 20$ already gives good results in terms of accuracy, at the cost of a low computational cost. While this energy saturation explains the rapid convergence for 'happy' and 'sleepy', it highlights that for 'left-light' expressions, the primary principal components contain significant lighting-related noise. This suggests that identifying a face under unfamiliar light exposure poses a substantial challenge, especially when such conditions are absent from the training set, causing the model to struggle even as more features are added. We will revisit this topic in a later section.

To investigate this hypothesis, we evaluated which expressions are most challenging for the model to recognize. We fixed the number of features at 20 (as the cumulative energy reaches approximately 90% at this threshold) and assessed the accuracy by systematically excluding each expression from the training set. The results confirmed our hypothesis: the lowest accuracy levels occurred when the 'left-light', 'right-light', or 'center-light' expressions were absent from the training data.

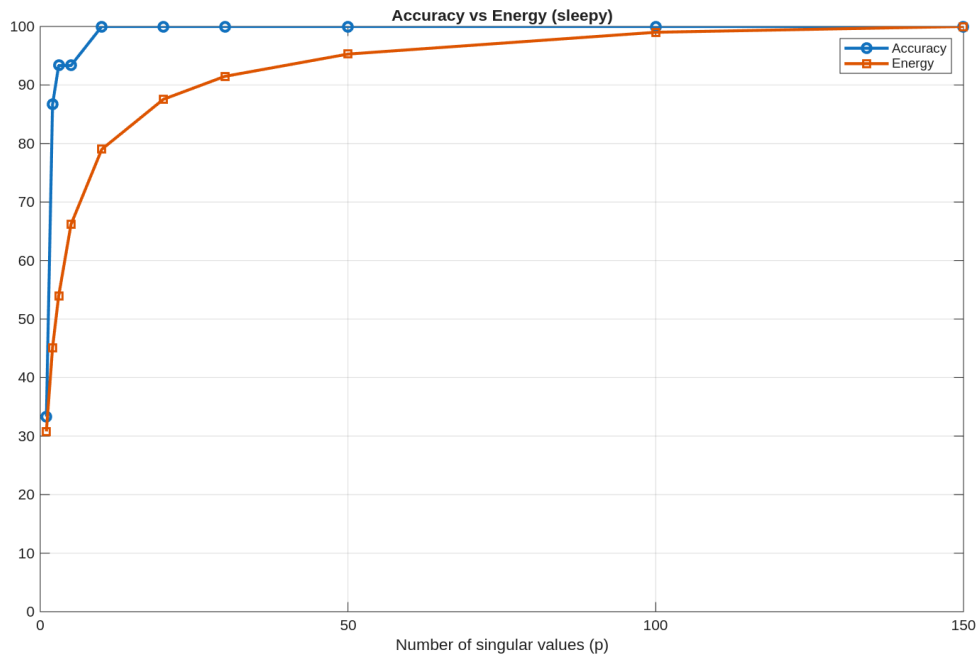


Figure 2: Accuracy and energy as a function of the number of features for sleepy expression.

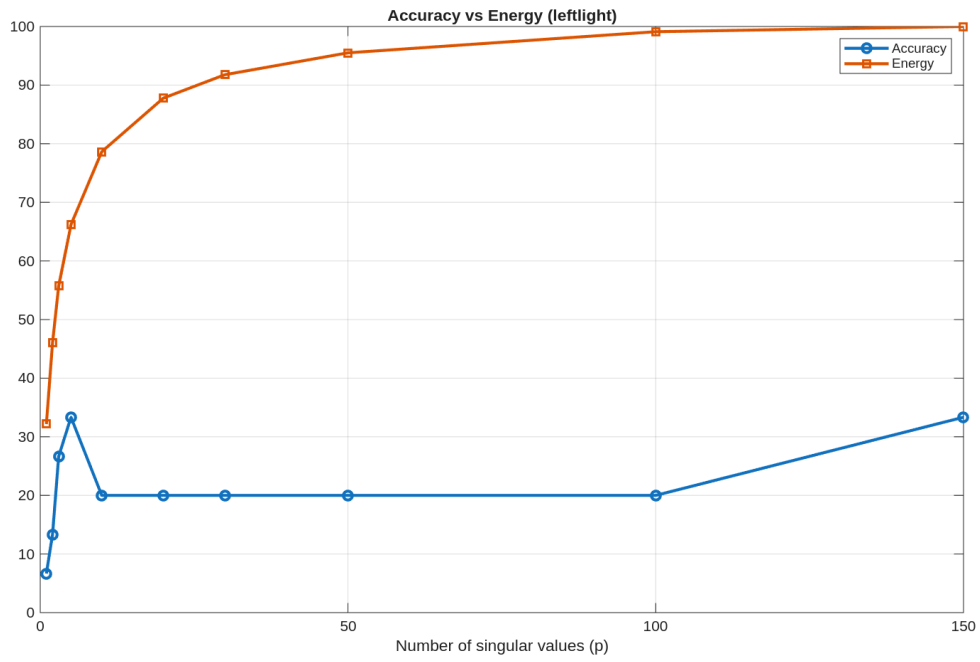


Figure 3: Accuracy and energy as a function of the number of features for leftlight expression.

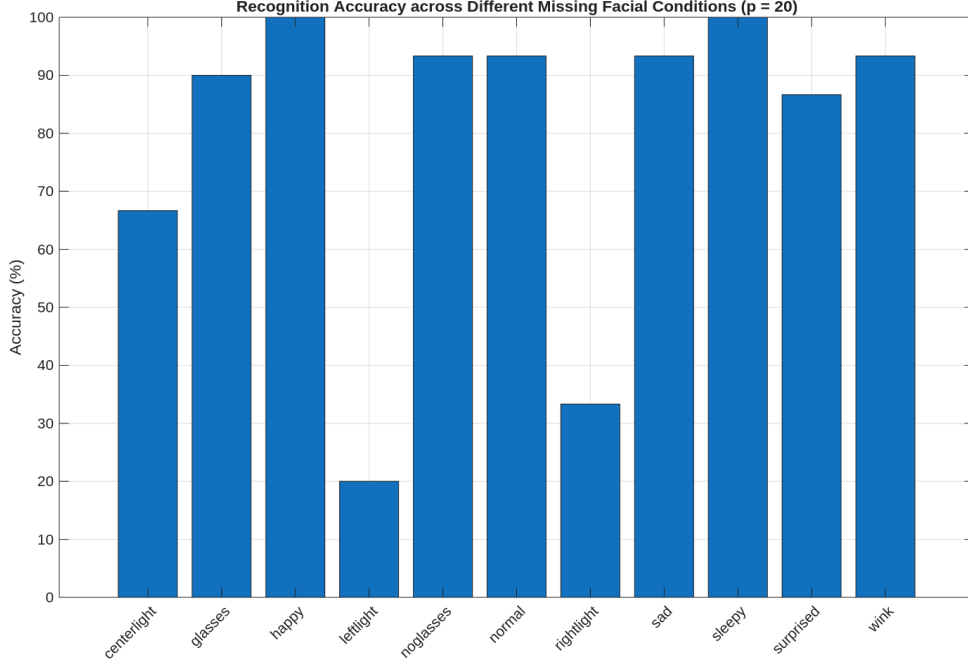


Figure 4: Recognition accuracy across different missing facial conditions ($p=20$).

3.1.2 Evaluation of model robustness against unknown subjects

We subsequently investigated the algorithm’s performance when presented with a novel subject not included in the training set. Specifically, we fixed the number of features at 20 and excluded ‘Subject 03’ from the dataset to test the recognition of an unknown individual. The experiment demonstrated that the minimum distances between the novel subject and the training set are generally significantly higher than those observed when a subject is correctly identified. To implement a robust face recognition system, a distance threshold could be established to determine whether a subject is recognized or rejected. To identify an appropriate threshold, we analyzed the minimum distances in the ‘sleepy’ expression experiment, where the model previously achieved 100% accuracy. In that scenario, the minimum distance averaged 151.18 ($SD = 117.12$), with a maximum value of 453.02. In contrast, the average minimum distance for an unknown subject was 515.56, with a minimum observed value of 429.73. By setting a threshold of 350, we can achieve a very low False Acceptance Rate (FAR) at the expense of a higher False Rejection Rate (FRR). This trade-off ensures a more secure and conservative face recognition system. Results of these experiments are shown in Figures 5 and 6.

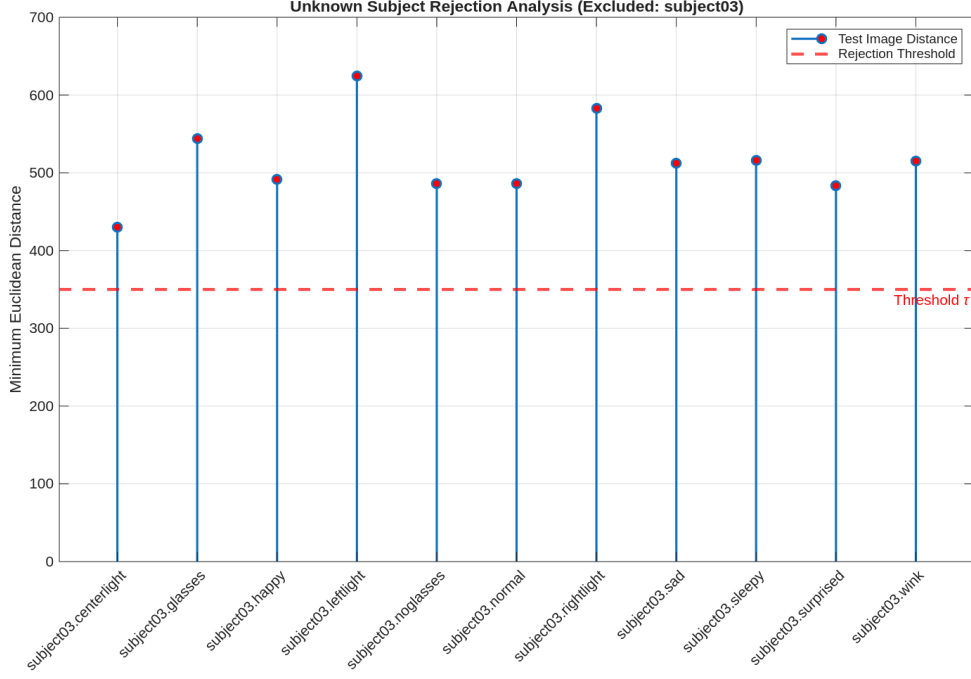


Figure 5: Minimum distances between the novel subject and the training set ($p=20$, Subject 03).

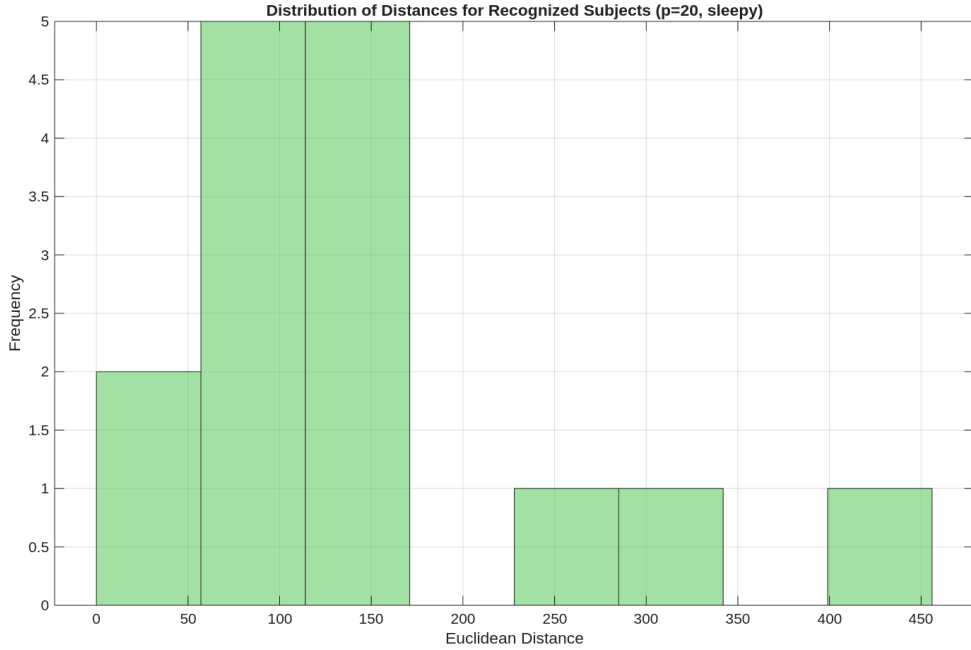


Figure 6: Distribution of distances for recognized subjects ($p=20$, sleepy).

3.1.3 Leave- k -Out Cross-Validation for the model

In the previous sections, the training sets were intentionally biased by the targeted exclusion of specific expressions or subjects. Our results indicated that performance is heavily dependent on the diversity of lighting conditions within the training data. To evaluate the model's predictive power in a more generalized and realistic context, we implemented a Leave- k -Out Cross-Validation (LkOCV) approach, in the case where the number of features is fixed at $p = 20$. In this setup, k samples are iteratively excluded from the training set and reserved for testing. We conducted experiments using two distinct values for k : $k = 15$ and $k = 50$. The choice

of $k = 15$ was specifically intended to maintain a training set size consistent with our previous 'leave-one-expression-out' tests, where one expression was omitted for each of the 15 subjects. By contrast, $k = 50$ was used to assess the model's robustness and performance decay when the available training data is significantly reduced. To ensure statistical reliability, the process was repeated over 200 iterations, from which we calculated the mean accuracy and standard deviation. Results are shown in Figures 7 and 8.

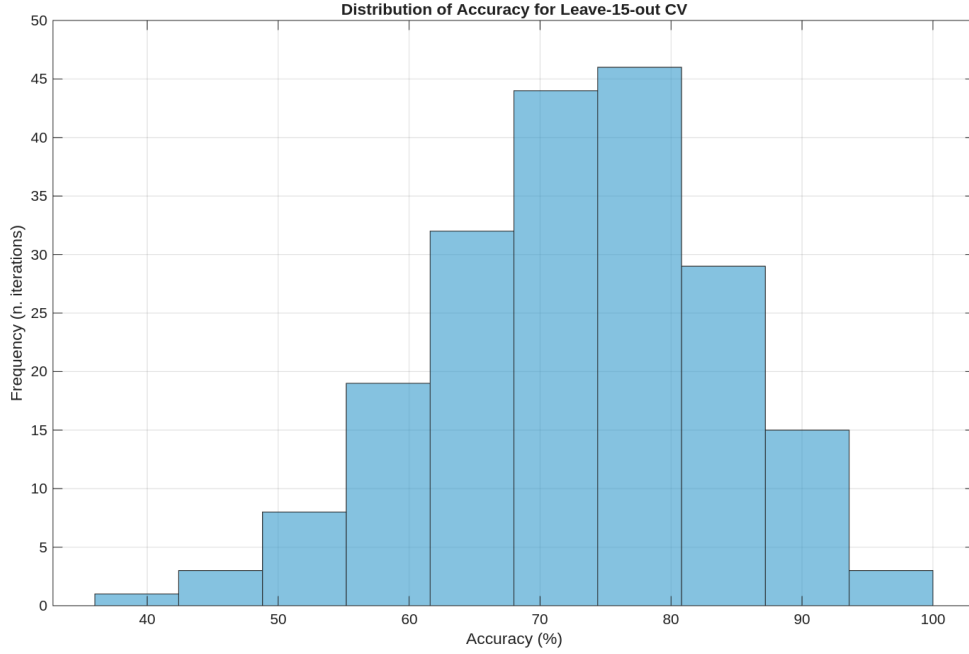


Figure 7: Leave- k -Out Cross-Validation ($p = 20$, $k = 15$).

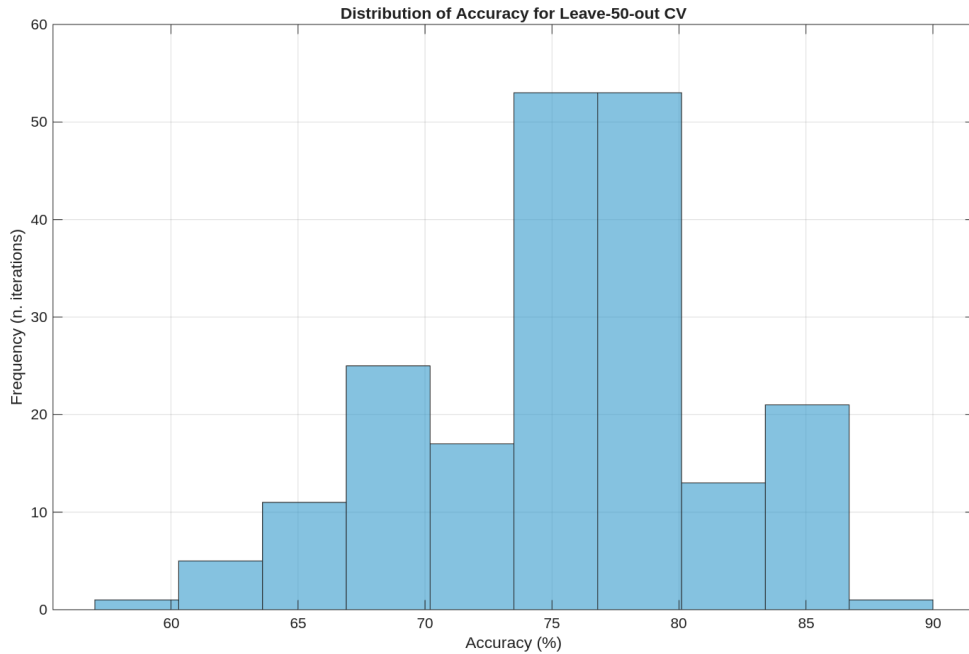


Figure 8: Leave- k -Out Cross-Validation ($p = 20$, $k = 50$).

For $k = 15$, the mean accuracy is 75.00% and the standard deviation is 11.44%, whereas for $k = 50$ the mean accuracy is 75.65% and the standard deviation is 5.86%. The results indicate that the model's performance does not degrade when the available training data is reduced. In-

terestingly, the cross-validation results show that increasing k from 15 to 50 leads to a significant reduction in variance. This phenomenon can be attributed to the representative nature of the validation sets. At $k = 15$, the small sample size makes the test performance highly sensitive to the specific images selected; iterations involving only 'standard' lighting yield disproportionately high scores, while those containing outliers or extreme lighting conditions result in sharp performance drops, thus increasing the variance. Conversely, at $k = 50$, the probability of including a balanced mix of both standard and challenging lighting conditions in every iteration is much higher. This statistical smoothing effect ensures that each test fold is more representative of the overall dataset's complexity, leading to more consistent results across the 200 iterations and explaining the lower variance observed in the $k = 50$ configuration.

3.2 Face reconstruction

What if we want to see the compressed face? We can reconstruct the face just by projecting the centered face onto the subspace spanned by the eigenfaces and then adding the mean face. For this experiment, we used the whole Yale Face Database as a training set. Results for 'subject01.happy' and 'subject01.rightlight' are shown in Figures 9 and 10.

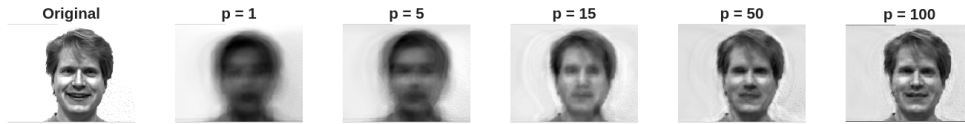


Figure 9: Face reconstruction for 'subject01.happy'.

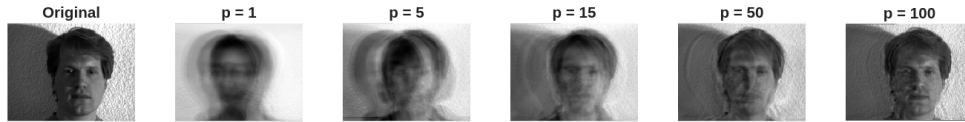


Figure 10: Face reconstruction for 'subject01.rightlight'.

As evidenced by the images, the number of features required for high-fidelity reconstruction is significantly greater than that needed for face recognition.

3.3 Signal and noise in facial images

From the experiments in the previous sections, we can see that lighting conditions play a crucial role in determining the accuracy of the model. We want to address this problem and reduce the noise given by lighting conditions.

References

- [1] L. Robol and S. Massei. *Appunti di Calcolo Scientifico*. 2025.
- [2] N. Muller, L. Magaia, and B. M. Herbst. "Singular Value Decomposition, Eigenfaces, and 3D Reconstructions". In: *SIAM Review* 46.3 (2004), pp. 518–545.
- [3] P. Belhumeur, J. Hespanha, and D. Kriegman. "Eigenfaces vs. Fisherfaces: Recognition Using Class Specific Linear Projection". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 19.7 (1997), pp. 711–720.